Machine Translation: From Rule-based Systems to Neural Transformers

Abstract

Machine Translation (MT) is the task of automatically converting text from one natural language to another. It has been a goal of computational linguistics since the 1940s, evolving through several paradigms. This paper provides an extended overview of MT's evolution, covering Rule-Based MT, Statistical MT (word-based and phrase-based models, alignment, decoding), Neural MT (sequence-to-sequence RNNs, attention mechanisms, Transformers), modern pretrained multilingual models, and common evaluation metrics (BLEU, METEOR, COMET, etc.). The presentation is aimed at graduate students, balancing historical context with technical detail.

1 Introduction

Machine Translation (MT) is the task of automatically converting text from one natural language to another. It has been a goal of computational linguistics since the 1940s, evolving through several paradigms. Early research in MT was inspired by pioneers like Warren Weaver, whose 1949 memorandum *Translation* is regarded as a foundational vision for the field [18]. Weaver speculated on various strategies for translation, such as using context to resolve ambiguity and even treating translation like a cryptography problem [18, 19].

Over the decades, MT approaches have shifted from hand-crafted rule-based systems to data-driven statistical methods, and most recently to end-to-end neural models. This chapter provides an extended overview of MT's evolution, covering Rule-Based MT, Statistical MT (word-based and phrase-based models, alignment, decoding), Neural MT (sequence-to-sequence RNNs, attention mechanisms, Transformers), modern pretrained multilingual models, and common evaluation metrics (BLEU, METEOR, COMET, etc.).

2 Rule-Based Machine Translation (RBMT)

2.1 Historical Background

The earliest MT systems were rule-based, built on linguistic rules and bilingual dictionaries. In the 1950s-60s, researchers attempted to encode grammatical structures and lexical mappings by hand. A famous early demonstration was the 1954 Georgetown experiment, which used a limited vocabulary and simplistic grammar rules to translate Russian sentences to English. This experiment successfully translated 60 Russian sentences into English using just 6 grammar rules and a 250-word vocabulary.

Rule-based systems often employed one of three approaches:

- **Direct translation**: Word-for-word replacement with minimal syntactic rearrangement
- **Transfer-based**: Using intermediate representations of linguistic structure
- **Interlingua-based**: Using a language-neutral meaning representation

Weaver's memorandum in 1949 had already cautioned against naive word-for-word translation and suggested leveraging context and universal linguistic principles [18, 19]. His vision spurred significant early MT efforts in the US and beyond.

2.2 Technical Approach

RBMT systems rely on extensive linguistic knowledge encoded in three main components:

2.2.1 Lexical Component

The lexical component consists of:

- Source language dictionary with morphological information
- Target language dictionary with generation rules
- Bilingual dictionary mapping source words to target words

Formally, a lexical entry might be represented as:

$$\text{entry}(w_s) = \{\text{lemma}, \text{POS}, \text{morphological features}, \{w_{t1}, w_{t2}, ...\}\} \tag{1}$$

Where w_s is a source word, and $\{w_{t1}, w_{t2}, ...\}$ is the set of possible target language translations.

2.2.2 Syntactic Analysis and Transformation Rules

The analysis process can be formalized as a series of transformations:

$$\text{sourceText} \xrightarrow{\text{morphological analysis}} \text{sourceTokens} \xrightarrow{\text{syntactic parsing}} \text{sourceTree} \tag{2}$$

Transfer rules map source language structures to target language structures:

$$\text{Rule}_i : \text{Pattern}_s \rightarrow \text{Pattern}_t \mid \text{Condition} \tag{3}$$

For example, a rule for English-to-Spanish adjective-noun ordering might be:

$$\text{NP}(\text{Adj}(X), \text{N}(Y)) \rightarrow \text{NP}(\text{N}(Y), \text{Adj}(X)) \tag{4}$$

This represents the transformation of English structure (adjective before noun) to Spanish structure (noun before adjective).

2.2.3 Generation Component

The final stage transforms the target language structure into properly inflected output:

$$\text{targetTree} \xrightarrow{\text{syntactic generation}} \text{targetTokens} \xrightarrow{\text{morphological generation}} \text{targetText} \tag{5}$$

2.3 Architectural Approaches

2.3.1 Direct Translation

The simplest approach performs word-for-word substitution with minimal reordering:

[node distance=2cm] (source) Source Text; [right of=source, xshift=1cm] (dict) Dictionary Lookup; [right of=dict, xshift=1cm] (local) Local Reordering; [right of=local, xshift=1cm] (target) Target Text; [-¿] (source) – (dict); [-¿] (dict) – (local); [-¿] (local) – (target);

Figure 1: Direct translation approach

Example:
English: "The boy eats the red apple"
Word-for-word Spanish: "El chico come la roja manzana"

Reordered Spanish: "El chico come la manzana roja"

The direct approach applies simple reordering rules (like moving adjectives after nouns for English-to-Spanish) but lacks deeper linguistic analysis.

2.3.2 Transfer-Based Approach

The most widely implemented RBMT architecture uses a transfer approach with three phases:

[node distance=2cm] (source) Source Text; [below of=source] (analysis) Analysis; [right of=analysis, xshift=2cm] (sstruct) Source Structure; [right of=sstruct, xshift=2cm] (transfer) Transfer; [right of=transfer, xshift=2cm] (tstruct) Target Structure; [right of=tstruct, xshift=2cm] (generation) Generation; [below of=generation] (target) Target Text; [-¿] (source) – (analysis); [-¿] (analysis) – (sstruct); [-¿] (sstruct) – (transfer); [-¿] (transfer) – (tstruct); [-¿] (tstruct) – (generation); [-¿] (generation) – (target);

Figure 2: Transfer-based translation approach

Example: Translating "The cat chased the mouse" from English to Japanese

Analysis phase:

$$\text{Parse("The cat chased the mouse")} \rightarrow \text{S(NP(DET(the), N(cat)), VP(V(chased), NP(DET(the), N(mo} \tag{6}$$

Transfer phase: Apply structural transfer rules:

$$\text{S(NP, VP(V, NP}_{\text{obj}}\text{))} \rightarrow \text{S(NP, NP}_{\text{obj}}\text{, V)} \tag{7}$$

This transforms the English SVO structure to Japanese SOV structure.

Generation phase: Apply morphological rules and produce:

$$\text{""} \text{ (Neko ga nezumi o oikaketa)} \tag{8}$$

2.3.3 Interlingua Approach

The most ambitious RBMT architecture translates source text into a language-independent semantic representation, then generates the target language from this representation:

Interlingua representation might use a semantic formalism like:

$$\text{CHASE(AGENT : CAT(DEF : +), PATIENT : MOUSE(DEF : +), TENSE : PAST)} \tag{9}$$

[node distance=2cm] (source) Source Text; [below of=source, xshift=-2cm] (analysis1) Analysis; [below of=source, xshift=2cm] (analysis2) Analysis; [below of=analysis1] (interlingua) Interlingua; [below of=analysis2] (generation) Generation; [below of=generation] (target) Target Text; [-¿] (source) – (analysis1); [-¿] (analysis1) – (interlingua); [-¿] (interlingua) – (generation); [-¿] (generation) – (target);

Figure 3: Interlingua translation approach

Advantage: For translating between multiple languages, only one analysis and one generation component is needed per language, rather than separate transfer components for each language pair.

Challenge: Creating a truly language-independent representation proved extremely difficult, particularly for divergent languages.

2.4 Real-World RBMT Systems

2.4.1 SYSTRAN

One of the earliest commercial MT systems, SYSTRAN, began as a rule-based system in the 1970s and was used by:

- The US Air Force for Russian-to-English translation
- The European Commission for translation between European languages
- Google's early translation service before they switched to statistical methods

2.4.2 METEO

A specialized system developed in Canada for translating weather forecasts between English and French:

- Extremely high accuracy within its limited domain
- Used successfully from 1977 to 2001
- Example: "Northwest winds 20 km/h" → "Vents du nord-ouest à 20 km/h"

2.4.3 Apertium

A modern open-source RBMT platform designed for related language pairs:

- Uses a shallow-transfer approach
- Particularly effective for closely related languages like Spanish-Catalan or Czech-Slovak

- Still actively developed and used today

2.5 Strengths and Limitations

2.5.1 Strengths

- **Predictable output**: Because rules are hand-crafted, translations are consistent and predictable
- **No training data required**: Can be built for language pairs with limited parallel text
- **Linguistic control**: Linguists can directly fix specific translation problems
- **Domain adaptation**: Rules can be tailored for specific domains (legal, medical, etc.)

Example of RBMT strength: For specialized domains with fixed terminology and structures, RBMT can achieve high accuracy:

Medical text: "The patient exhibits elevated levels of serum glucose."
RBMT with medical dictionary: "El paciente muestra niveles elevados de glucosa en suero."

2.5.2 Limitations

- **Labor-intensive development**: Creating comprehensive rule sets requires years of work by skilled linguists
- **Rule conflicts**: As rule sets grow, interactions between rules become increasingly complex and unpredictable
- **Coverage problems**: Impossible to cover all linguistic phenomena with explicit rules
- **Difficulty with ambiguity**: Rules struggle with lexical and structural ambiguities
- **Maintenance complexity**: Adding new rules or vocabulary often requires revising existing rules

Example of RBMT limitation: The sentence "Time flies like an arrow" has multiple valid syntactic analyses:

- "Time" as noun, "flies" as verb: [The passage of time] [moves quickly] [like an arrow does]
- "Time flies" as noun phrase: [A type of fly called 'time flies'] [enjoy] [an arrow]

- "Time" as verb, imperative: [Measure the speed of] [flies] [the same way you would for an arrow]

Without world knowledge, selecting the correct interpretation for translation is extremely difficult for rule-based systems.

2.6 Transition to Statistical Approaches

As more bilingual text became available and computing power grew, researchers sought data-driven approaches that could learn translation patterns automatically. This led to the development of Statistical Machine Translation (SMT), which addressed several limitations of RBMT:

- Instead of manually coding rules, SMT automatically extracts translation patterns from data
- Ambiguities are resolved through probabilistic models rather than deterministic rules
- SMT systems can learn idiomatic expressions and exceptions naturally from examples

The ALPAC report of 1966, which criticized early rule-based MT efforts as expensive and ineffective, temporarily slowed MT research. However, by the late 1980s, increased computing power and the availability of parallel corpora enabled the development of SMT, marking a paradigm shift away from purely rule-based approaches.

2.7 Legacy and Modern Applications

Despite being largely superseded by statistical and neural approaches, rule-based techniques remain relevant in several contexts:

- **Hybrid systems**: Many commercial MT systems combine rule-based components with statistical or neural models
- **Low-resource languages**: For language pairs with minimal parallel data, rule-based components are still valuable
- **Controlled domains**: In highly specialized fields with strict terminology requirements, rule-based components ensure consistency

Rules also continue to play a role in pre-processing and post-processing steps even in modern neural MT systems, demonstrating the enduring value of linguistic knowledge in machine translation.

3 Statistical Machine Translation (SMT)

Statistical Machine Translation was introduced in the late 1980s and early 1990s as a radically different approach: instead of manual rules, the system learns to translate by analyzing large parallel corpora (source sentences and their human translations). SMT treats translation as a probabilistic inference problem. A translation system is trained to maximize the probability of a target sentence given a source sentence, using statistical models whose parameters are learned from data.

3.1 Word-Based SMT and IBM Models 1–5

The first influential SMT models were word-based. A team at IBM pioneered this approach, introducing a series of probabilistic models (IBM Models 1 through 5) in a seminal paper by Brown et al. [3]. These models establish the concept of word alignment – hidden correspondences between words in the source and target sentences.

In the statistical approach to translation, we seek to model the probability distribution $P(f|e)$ where f is a foreign (source) sentence and e is an English (target) sentence. Using Bayes' rule, this can be rewritten as:

$$P(e|f) = \frac{P(f|e)P(e)}{P(f)} \propto P(f|e)P(e) \tag{10}$$

Where $P(e)$ is the language model probability and $P(f|e)$ is the translation model probability. The IBM models focus on developing sophisticated models for $P(f|e)$.

In IBM Models 1–5, translation is modeled as a generative process where each source word can produce one or more target words (or none). For example, the model might learn that the French word "maison" aligns to the English word "house" with high probability. The IBM models progressively increase in complexity:

- **Model 1:** Each source word independently translates to a target word according to a lexical probability table. It assumes no word-order distortion (word positions are ignored).

  The probability of a French sentence $f = (f_1, f_2, \ldots, f_m)$ given an English sentence $e = (e_1, e_2, \ldots, e_l)$ and sentence lengths m and l is:

  $$P(f|e) = \frac{\epsilon}{(l+1)^m} \prod_{j=1}^{m} \sum_{i=0}^{l} t(f_j|e_i) \tag{11}$$

  Where $t(f_j|e_i)$ is the lexical translation probability of source word f_j given target word e_i. The term e_0 is a special NULL token to account for words that have no corresponding translation. The factor

$\frac{\epsilon}{(l+1)^m}$ represents the uniform alignment probability, where each possible alignment is equally likely.

- **Model 2:** Adds a simple alignment model to account for positional alignment by introducing an alignment probability distribution $a(i|j,l,m)$ representing the probability that the j-th source word aligns to the i-th target position, given sentence lengths l and m:

$$P(f|e) = \epsilon \prod_{j=1}^{m} \sum_{i=0}^{l} a(i|j,l,m) \cdot t(f_j|e_i) \tag{12}$$

  This model can capture basic positional preferences (e.g., that words at the beginning of the source sentence tend to align with words at the beginning of the target sentence).

- **Model 3:** Introduces fertility $n(|e_i)$, the probability that target word e_i generates source words. Let a_j be the position in e that generates f_j. The probability becomes:

$$P(f,a|e) = \prod_{i=0}^{l} \binom{\phi_i}{\phi_{i1}, \phi_{i2}, \ldots, \phi_{iC}} \prod_{j=1}^{m} t(f_j|e_{a_j}) \cdot d(j|a_j,l,m) \cdot \prod_{i=0}^{l} n(\phi_i|e_i) \tag{13}$$

  Where ϕ_i is the fertility of word e_i (how many source words it generates), and $d(j|i,l,m)$ is the distortion probability (replacing the simpler alignment model from Model 2) that a source word at position j is produced by a target word at position i. The multinomial coefficient accounts for different ways to choose which positions around e_i produce which source words.

- **Model 4:** Refines alignment with a relative distortion model that handles reordering based on word classes. Instead of absolute positions, it models the probability of placement relative to the position of the previous source word generated from the same target word class:

$$P(f,a|e) = \prod_{i=0}^{l} n(\phi_i|e_i) \prod_{j:a_j=i} t(f_j|e_i) \cdot d_1(j - c_{\text{prev}(j)}|\mathcal{C}(e_i), \mathcal{C}(f_j)) \tag{14}$$

  Where $\mathcal{C}(w)$ is the word class of word w, $c_{\text{prev}(j)}$ is the position of the previously generated source word, and d_1 is a distortion probability conditioned on word classes, modeling relative positions rather than absolute ones.

- **Model 5:** Fixes deficiencies in Model 4 (such as the possibility of two source words aligning to the same position) by adding more parameters to ensure that each source position is filled exactly once. It uses a "vacancy" mechanism to track available positions in the source sentence:

$$P(f, a|e) = \prod_{i=0}^{l} n(\phi_i|e_i) \prod_{j:a_j=i} t(f_j|e_i) \cdot d_2(j|\text{vacancies}(j), \mathcal{C}(e_i), \mathcal{C}(f_j)) \tag{15}$$

Where d_2 is a modified distortion probability that depends on available vacant positions, ensuring no overlaps in source word generation.

3.1.1 Alignment and EM Algorithm

These models use the Expectation-Maximization (EM) algorithm to learn their parameters from parallel corpora. For Model 1, the update rule for translation probabilities in each EM iteration is:

$$t(f|e) = \frac{\sum_{(e,f)\in\text{corpus}} \sum_{j:f_j=f} \sum_{i:e_i=e} P(a_j = i|e, f)}{\sum_{(e,f)\in\text{corpus}} \sum_{j} \sum_{i:e_i=e} P(a_j = i|e, f)} \tag{16}$$

Where $P(a_j = i|e, f)$ is the posterior probability of alignment given the current parameter estimates and can be calculated as:

$$P(a_j = i|e, f) = \frac{t(f_j|e_i)}{\sum_{i'=0}^{l} t(f_j|e_{i'})} \tag{17}$$

For higher-numbered models, the EM update equations become significantly more complex, as they need to account for fertility and distortion parameters.

Brown et al.'s work framed translation in a rigorous probabilistic way and provided algorithms to learn these parameters from sentence-aligned bilingual corpora. The IBM models demonstrated that a computer could "learn" translation lexicons and alignments from data, a significant departure from laborious manual rule creation.

These word-based models were the foundation of SMT for about two decades. In practice, they were used to produce word alignments, and the alignments would then inform more advanced models. Notably, Vogel et al. [17] introduced an HMM alignment model as an alternative between IBM Model 2 and 3:

$$P(f, a|e) = \prod_{j=1}^{m} t(f_j|e_{a_j}) \cdot p(a_j|a_{j-1}, l) \tag{18}$$

Where $p(a_j|a_{j-1}, l)$ is a first-order Markov probability of an alignment position given the previous alignment position, capturing local ordering phenomena more effectively than Model 2 but with less complexity than Model 3.

Despite their simplifications (e.g., initially assuming independence between word translations), the IBM models underpinned most MT systems until the 2010s. Many later systems still start by training IBM Models 1–4 (as implemented in tools like GIZA++) to get initial alignments, even when using more sophisticated phrase-based or neural approaches for the final translation.

3.1.2 Word Alignment

The goal of alignment is to find which source word generated each target word. Formally, for a source sentence $f = (f_1, f_2, \ldots, f_m)$ and a target sentence $e = (e_1, e_2, \ldots, e_l)$, an alignment $a = (a_1, a_2, \ldots, a_m)$ specifies that source word f_j is aligned to target word e_{a_j} (or to NULL if $a_j = 0$).

English:	It	will	surely	rain	tomorrow
	↓		↓	↓	↓
Japanese:		(tomorrow)	(surely)	(rain)	(is)

Figure 4: Example word alignment between English and Japanese sentences. Note the reordering of words and that "It" and "will" have no direct alignment, while "" (topic marker) aligns to NULL.

In this example, alignments link "surely" → "", "rain" → "", "tomorrow" → "", while some words (like English "it" or Japanese particles) remain unaligned or align to a null placeholder.

The IBM models treat alignment as a latent variable a, and we seek to maximize the likelihood of the observed parallel corpus:

$$P(f|e) = \sum_a P(f, a|e) \tag{19}$$

Since direct maximization is intractable due to the summation over all possible alignments, the EM algorithm is used:

E-step: Compute expected counts for each parameter based on current estimates:

$$c(f|e) = \sum_{(e,f)} \sum_{j:f_j=f} \sum_{i:e_i=e} P(a_j = i|e, f) \tag{20}$$

M-step: Re-estimate parameters to maximize likelihood given these counts:

$$t(f|e) = \frac{c(f|e)}{\sum_{f'} c(f'|e)} \tag{21}$$

For Model 1, the posterior alignment probability in the E-step is:

$$P(a_j = i|e, f) = \frac{t(f_j|e_i)}{\sum_{i'=0}^{l} t(f_j|e_{i'})} \tag{22}$$

This probability distribution over alignments is used both for training and for finding the most probable alignment at test time:

$$\hat{a} = \arg\max_a P(a|e, f) = \arg\max_a P(f, a|e) \tag{23}$$

For Model 1, this results in a simple decision rule:

$$\hat{a}_j = \arg\max_i t(f_j|e_i) \tag{24}$$

Model 1's parameter estimation is convex, meaning it will converge to the same solution regardless of initialization. This makes it mathematically attractive and is why it's typically used to initialize parameters for more complex models.

For higher-numbered models (IBM Model 3+), finding the most probable alignment becomes NP-hard. The search space grows exponentially with sentence length, as there are $(l+1)^m$ possible alignments between an l-word target and m-word source sentence. The problem can be formulated as:

$$\hat{a} = \arg\max_a \prod_{j=1}^{m} t(f_j|e_{a_j}) \cdot d(j|a_j, m, l) \cdot \prod_{i=0}^{l} n(\phi_i|e_i) \tag{25}$$

Due to this complexity, approximate search algorithms are used:

1. **Greedy hill-climbing**: Start with Model 1/2 alignments and make local changes to increase probability

2. **Beam search**: Maintain a list of k best partial alignments while building the solution

3. **Gibbs sampling**: Sample alignments from the posterior distribution and take the maximum probability alignment found

In practice, bidirectional alignment combination techniques are often used to improve quality. By training separate models for source→target and target→source directions, then combining the alignments through intersection, union, or more sophisticated heuristics like grow-diag-final, more reliable alignments can be obtained:

$$A_{final} = f(A_{f\text{ß}e}, A_{e\text{ß}f}) \tag{26}$$

These refined alignments serve as the foundation for extracting phrase pairs in phrase-based SMT systems or creating training data for neural attention mechanisms in NMT.

3.2 Phrase-Based SMT

By the early 2000s, it became clear that translating individual words was not sufficient. Phrase-Based SMT extended the atomic unit of translation from single words to sequences of words ("phrases" in this context are contiguous word sequences, not necessarily linguistic phrases). The phrase-based approach was spearheaded by researchers like Franz Och, Philipp Koehn, and Daniel Marcu. Koehn et al. [9] introduced the first full phrase-based translation model, showing that using phrases greatly improved translation quality compared to word-based models.

3.2.1 Key Ideas and Technical Formulation

In phrase-based SMT, the translation probability of a foreign sentence f to an English sentence e is decomposed into three main components:

$$P(e|f) \propto P(f|e) \cdot P(e) \cdot P_d(alignment) \tag{27}$$

Where:

- $P(f|e)$ is the translation model probability
- $P(e)$ is the language model probability
- $P_d(alignment)$ represents the reordering/distortion probability

Phrase Extraction: Given word alignments A between sentences (e, f), a phrase pair $(\bar{e}, \bar{f})$ is extracted if all words in $\bar{e}$ align only to words in $\bar{f}$ and vice versa:

$$\forall e_i \in \bar{e}, f_j \in \bar{f} : (i,j) \in A \wedge \forall e_i \in \bar{e} : (i,j') \in A \Rightarrow f_{j'} \in \bar{f} \wedge \forall f_j \in \bar{f} : (i',j) \in A \Rightarrow e_{i'} \in \bar{e} \tag{28}$$

Example: For the aligned sentence pair:

I	saw	the	small	house	
↓	↓	↓	↓	↓	
Je	ai	vu	la	petite	maison

Valid phrase pairs include ("the small house", "la petite maison"), ("small house", "petite maison"), but not ("the small", "la petite") because "house" aligns with "maison".

Phrase Translation Probability: For a phrase pair $(\bar{e}, \bar{f})$, the translation probability is estimated by relative frequency:

$$\phi(\bar{f}|\bar{e}) = \frac{\text{count}(\bar{e}, \bar{f})}{\sum_{\bar{f}'} \text{count}(\bar{e}, \bar{f}')} \tag{29}$$

Lexical Weighting: To address sparsity, phrase probabilities are smoothed with word-level translation probabilities:

$$p_{lex}(\bar{f}|\bar{e}, a) = \prod_{i=1}^{|\bar{f}|} \frac{1}{|\{j|(i,j) \in a\}|} \sum_{\forall (i,j) \in a} w(f_i|e_j) \tag{30}$$

Where $w(f_i|e_j)$ is the lexical translation probability from IBM Models.

3.2.2 Log-Linear Model for Decoding

The complete model combines multiple features in a log-linear framework:

$$\text{score}(e, f) = \exp\left(\sum_{i=1}^{n} \lambda_i h_i(e, f)\right) \tag{31}$$

Key feature functions $h_i(e, f)$ include:

- Phrase translation probabilities: $\log \phi(\bar{f}|\bar{e})$ and $\log \phi(\bar{e}|\bar{f})$
- Lexical weighting: $\log p_{lex}(\bar{f}|\bar{e})$ and $\log p_{lex}(\bar{e}|\bar{f})$
- Language model: $\log P(e)$
- Word/phrase penalties to control output length
- Distortion: $d(start_i - end_{i-1} - 1)$ penalizing deviation from monotonic order

Weights λ_i are optimized on a development set to maximize translation quality metrics (typically BLEU).

3.2.3 Decoding Algorithm

The decoding algorithm searches for:

$$\hat{e} = \underset{e}{\text{argmax}} \sum_{i=1}^{n} \lambda_i h_i(e, f) \tag{32}$$

Since this search is NP-hard, beam search is used:
Each hypothesis tracks:

- Translated words (coverage vector)
- Partial translation generated so far
- Last $n - 1$ target words (for n-gram language model)
- Current score and estimated future cost

Algorithm 1 Phrase-Based Decoder with Beam Search

```
Initialize empty hypothesis with zero score
beams[0] = {emptyHypothesis}
for end = 1 to |f| do
    for start = end downto max(1, end − maxPhraseLength) do
        sourcePhrase = f[start : end]
        for targetPhrase in phraseTable[sourcePhrase] do
            for hyp in beams[start − 1] do
                if hyp can be extended with targetPhrase then
                    Create new hypothesis newHyp
                    Add translation and reordering costs
                    Add newHyp to appropriate beam
                end if
            end for
        end for
    end for
    Prune beams to keep only top k hypotheses
end for
return highest scoring complete hypothesis
```

3.2.4 Example Decoding Process

Input: "Ich möchte ein Bier"

Phrase table entries:

Source	Target	Score
Ich	I	-0.5
möchte	want	-0.7
möchte	would like	-0.6
ein Bier	a beer	-0.3
ein	a	-0.8
Bier	beer	-0.9

Beam search (simplified):

1. Start with empty hypothesis [coverage: 0000, text: "", score: 0]
2. Expand with "Ich" → [coverage: 1000, text: "I", score: -0.5]
3. Expand with "möchte" → [coverage: 1100, text: "I want", score: -1.2] or [coverage: 1100, text: "I would like", score: -1.1]
4. Continue highest-scoring path, expand with "ein Bier" → [coverage: 1111, text: "I would like a beer", score: -1.4]

Final translation: "I would like a beer"

3.3 Decoding and Search in SMT

The decoding problem in SMT can be formally defined as:

$$\hat{e} = \underset{e}{\operatorname{argmax}}\, P(e|f) = \underset{e}{\operatorname{argmax}}\, P(f|e)P(e) \tag{33}$$

For phrase-based systems, this becomes a search over possible segmentations of the source, possible phrase translations, and possible reorderings - a massive combinatorial space.

Coverage vector: A bit vector C where $C_i = 1$ if word f_i has been translated, and 0 otherwise.

Future cost estimation: For partial hypothesis h with coverage C, the estimated cost to complete is:

$$FC(h) = \sum_{i=1}^{|f|} (1 - C_i) \cdot \min_{\text{phrases covering } f_i} \text{cost(phrase)} \tag{34}$$

This provides an admissible heuristic for pruning unpromising paths.

Distortion limit: To make search tractable, reordering is typically limited:

$$|start_i - end_{i-1} - 1| \leq d_{max} \tag{35}$$

Where $start_i$ is the position of the first word of the source phrase being translated, and end_{i-1} is the position of the last word of the previously translated phrase.

The limitations of SMT's piecewise optimization (separate training of translation, language, and reordering models) and the complexity of decoding eventually motivated the shift to neural approaches offering end-to-end optimization.

4 Neural Machine Translation (NMT)

Around 2014, the field of Machine Translation underwent a revolution with the advent of Neural Machine Translation. NMT uses neural networks to model the translation process in an end-to-end fashion, eliminating the need for separate subcomponents like alignment models, phrase tables, and language models. Instead, a single neural model (typically an encoder-decoder architecture) is trained to take a source sentence and directly produce its translation. This approach was made feasible by advances in deep learning, particularly recurrent neural networks with gating mechanisms designed to handle sequential data.

4.1 Sequence-to-Sequence RNNs

The first NMT models were based on recurrent neural networks (RNNs). In late 2014, two influential papers – by Cho et al. [5] and Sutskever et al. [14] – independently demonstrated that an encoder-decoder RNN could achieve credible translation results.

In the sequence-to-sequence framework introduced by Sutskever et al. [14], the probability of a target sequence $\mathbf{y} = (y_1, y_2, ..., y_T)$ given a source sequence $\mathbf{x} = (x_1, x_2, ..., x_S)$ is modeled as:

$$P(\mathbf{y}|\mathbf{x}) = \prod_{t=1}^{T} P(y_t|\mathbf{x}, y_1, y_2, ..., y_{t-1}) \tag{36}$$

This equation shows that we predict each target word y_t based on the source sentence $\mathbf{x}$ and all previously generated target words. This is implemented using two RNNs:

1. Encoder RNN: Processes the source sentence to create a fixed-length context vector $\mathbf{c}$:

$$\mathbf{h}_s = f(\mathbf{h}_{s-1}, \mathbf{x}_s) \quad \text{for } s = 1, 2, ..., S \tag{37}$$

$$\mathbf{c} = \mathbf{h}_S \tag{38}$$

Where $\mathbf{h}_s$ is the hidden state at source position s, and f is a non-linear function (typically LSTM or GRU). The final hidden state $\mathbf{h}_S$ becomes the context vector $\mathbf{c}$ that summarizes the entire source sentence.

2. Decoder RNN: Generates the target sentence one word at a time:

$$\mathbf{h}_t = g(\mathbf{h}_{t-1}, y_{t-1}, \mathbf{c}) \tag{39}$$

$$P(y_t|\mathbf{x}, y_1, ..., y_{t-1}) = \text{softmax}(\mathbf{W}\mathbf{h}_t + \mathbf{b}) \tag{40}$$

Where $\mathbf{h}_t$ is the hidden state at target position t, g is another non-linear function, $\mathbf{W}$ and $\mathbf{b}$ are learned parameters, and the softmax function converts the output to a probability distribution over the target vocabulary.

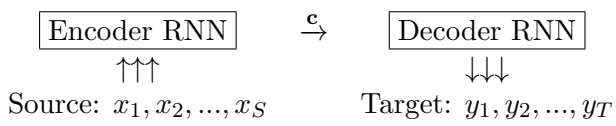

Figure 5: Basic sequence-to-sequence architecture for NMT.

In their original implementation, Sutskever et al. [14] used Long Short-Term Memory (LSTM) cells for both encoder and decoder. The LSTM addresses the vanishing gradient problem in traditional RNNs with its cell state $\mathbf{c}_t$ and hidden state $\mathbf{h}_t$ updated through three gates:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, x_t] + \mathbf{b}_f) \quad \text{(forget gate)} \tag{41}$$
$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, x_t] + \mathbf{b}_i) \quad \text{(input gate)} \tag{42}$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, x_t] + \mathbf{b}_o) \quad \text{(output gate)} \tag{43}$$
$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \cdot [\mathbf{h}_{t-1}, x_t] + \mathbf{b}_c) \quad \text{(candidate cell state)} \tag{44}$$
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad \text{(cell state update)} \tag{45}$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{(hidden state)} \tag{46}$$

Where σ is the sigmoid function, $\odot$ represents element-wise multiplication, and $[\mathbf{h}_{t-1}, x_t]$ denotes concatenation of the previous hidden state and current input.

Key innovation - Sentence reversal: Sutskever et al. found that reversing the source sentence (processing "I am a student" as "student a am I") improved performance significantly. This is because:

1. It creates shorter dependencies between corresponding words in source and target 2. The first few words in the source sentence (now at the end of the reversed sequence) are closer to the decoder, reducing the path for gradient flow 3. The reversal helps align the beginning of the target sentence with the relevant part of the context vector

4.1.1 Gated Recurrent Unit (GRU)

Cho et al. [5] introduced a simpler alternative to LSTM called the Gated Recurrent Unit (GRU). The GRU combines the forget and input gates into a single "update gate" and merges the cell state and hidden state:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \cdot [\mathbf{h}_{t-1}, x_t]) \quad \text{(update gate)} \tag{47}$$
$$\mathbf{r}_t = \sigma(\mathbf{W}_r \cdot [\mathbf{h}_{t-1}, x_t]) \quad \text{(reset gate)} \tag{48}$$
$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W} \cdot [\mathbf{r}_t \odot \mathbf{h}_{t-1}, x_t]) \quad \text{(candidate state)} \tag{49}$$
$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad \text{(hidden state update)} \tag{50}$$

The GRU is computationally more efficient than LSTM while achieving similar performance in many tasks.

4.1.2 Performance and Limitations

Early sequence-to-sequence models demonstrated that neural approaches could reach comparable performance to phrase-based SMT systems. However, they suffered from a fundamental constraint: the entire meaning of the source sentence had to be compressed into a single fixed-length vector $\mathbf{c}$. This bottleneck became particularly problematic for long sentences, where important information would get lost or diluted in the encoding process.

4.2 Attention Mechanisms in NMT

The breakthrough that alleviated the fixed-context bottleneck was the attention mechanism, introduced by Bahdanau et al. [1]. Their key insight was that the decoder should be able to focus on different parts of the source sentence at each decoding step, rather than relying on a single context vector.

4.2.1 How Attention Works

In the attention-based model, the encoder produces a sequence of hidden states $(\mathbf{h}_1, \mathbf{h}_2, ..., \mathbf{h}_S)$ rather than just using the final state. At each decoding step t, the decoder computes a different context vector $\mathbf{c}_t$ by taking a weighted sum of all encoder hidden states:

$$\mathbf{c}_t = \sum_{s=1}^{S} \alpha_{ts} \mathbf{h}_s \tag{51}$$

$$\alpha_{ts} = \frac{\exp(e_{ts})}{\sum_{s'=1}^{S} \exp(e_{ts'})} \tag{52}$$

$$e_{ts} = a(\mathbf{s}_{t-1}, \mathbf{h}_s) \tag{53}$$

Where:

- α_{ts} is the attention weight: how much the decoder at time t should focus on encoder state s
- e_{ts} is the alignment score computed by the alignment model a
- $\mathbf{s}_{t-1}$ is the decoder's previous hidden state

In Bahdanau's model, the alignment function a was a small feedforward neural network:

$$a(\mathbf{s}_{t-1}, \mathbf{h}_s) = \mathbf{v}_a^\top \tanh(\mathbf{W}_a[\mathbf{s}_{t-1}; \mathbf{h}_s]) \tag{54}$$

Where $\mathbf{v}_a$ and $\mathbf{W}_a$ are learned parameters, and $[\mathbf{s}_{t-1}; \mathbf{h}_s]$ denotes concatenation.

The decoder then uses this context vector along with its previous hidden state and the previous output word to generate the next word:

$$\mathbf{s}_t = f(\mathbf{s}_{t-1}, y_{t-1}, \mathbf{c}_t) \tag{55}$$

$$P(y_t|\mathbf{y}_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_o[\mathbf{s}_t; \mathbf{c}_t; y_{t-1}] + \mathbf{b}_o) \tag{56}$$

Example: When translating the English sentence "The cat sits on the mat" to French, when generating the word "tapis" (mat), the attention mechanism will assign high weights to the encoder hidden state corresponding to "mat" in the source sentence. This creates an implicit alignment between source and target words.

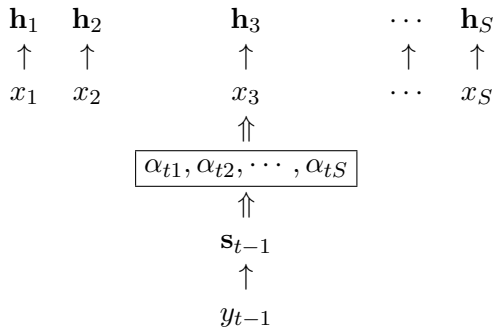

Figure 6: Attention mechanism computing alignment weights α_{ts} between decoder state $\mathbf{s}_{t-1}$ and all encoder states.

4.2.2 Variations of Attention

Luong et al. [11] proposed several alternative attention formulations:

1. **Dot product attention:** $e_{ts} = \mathbf{s}_{t-1}^\top \mathbf{h}_s$
2. **General attention:** $e_{ts} = \mathbf{s}_{t-1}^\top \mathbf{W}_a \mathbf{h}_s$
3. **Concatenation (similar to Bahdanau):** $e_{ts} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a[\mathbf{s}_{t-1}; \mathbf{h}_s])$

They also introduced a distinction between global attention (attending to all source words) and local attention (focusing on a small window of the source), with the latter being more efficient for long sequences.

4.3 The Transformer Model

The next major advancement came with the Transformer architecture introduced by Vaswani et al. [16] in 2017. The Transformer dispensed with recurrence entirely, relying exclusively on attention mechanisms to model sequential dependencies. This made training much more parallelizable and efficient.

4.3.1 Self-Attention Mechanism

The core innovation in Transformers is the self-attention mechanism, which allows each position in a sequence to attend to all positions in the same sequence:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{57}$$

Where $\mathbf{Q}$ (query), $\mathbf{K}$ (key), and $\mathbf{V}$ (value) are matrices derived from the

input sequence by linear projections:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q \tag{58}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}^K \tag{59}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}^V \tag{60}$$

The scaling factor $\sqrt{d_k}$ (where d_k is the dimension of the keys) prevents the softmax function from having extremely small gradients when the dot products grow large.

4.3.2 Multi-Head Attention

Instead of performing a single attention function, the Transformer employs multi-head attention, which allows the model to attend to information from different representation subspaces:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O \tag{61}$$

$$\text{where head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V) \tag{62}$$

Each "head" is an independent attention mechanism with its own learned projection matrices $\mathbf{W}_i^Q$, $\mathbf{W}_i^K$, and $\mathbf{W}_i^V$. The outputs of all heads are concatenated and linearly transformed with $\mathbf{W}^O$.

4.3.3 Architecture Overview

Complete Transformer Architecture:

The Transformer consists of an encoder and a decoder, each with a stack of N identical layers (typically $N = 6$):

Encoder Layer:

$$\hat{\mathbf{X}} = \text{LayerNorm}(\mathbf{X} + \text{MultiHeadAttention}(\mathbf{X}, \mathbf{X}, \mathbf{X})) \tag{63}$$

$$\mathbf{Z} = \text{LayerNorm}(\hat{\mathbf{X}} + \text{FFN}(\hat{\mathbf{X}})) \tag{64}$$

Where FFN is a position-wise feed-forward network with ReLU activation:

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 \tag{65}$$

Decoder Layer:

$$\hat{\mathbf{Y}} = \text{LayerNorm}(\mathbf{Y} + \text{MaskedMultiHeadAttention}(\mathbf{Y}, \mathbf{Y}, \mathbf{Y})) \tag{66}$$

$$\hat{\mathbf{Y}}' = \text{LayerNorm}(\hat{\mathbf{Y}} + \text{MultiHeadAttention}(\hat{\mathbf{Y}}, \mathbf{Z}, \mathbf{Z})) \tag{67}$$

$$\mathbf{O} = \text{LayerNorm}(\hat{\mathbf{Y}}' + \text{FFN}(\hat{\mathbf{Y}}')) \tag{68}$$

The "Masked" attention in the decoder prevents positions from attending to subsequent positions, maintaining the autoregressive property needed for generation.

4.3.4 Positional Encoding

Since the Transformer contains no recurrence or convolution, it has no inherent notion of sequence order. To inject information about position, the model adds sinusoidal positional encodings to the input embeddings:

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{69}$$

$$\mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{70}$$

Where pos is the position in the sequence and i is the dimension. These encodings have the useful property that relative positions can be represented by linear functions of the encodings.

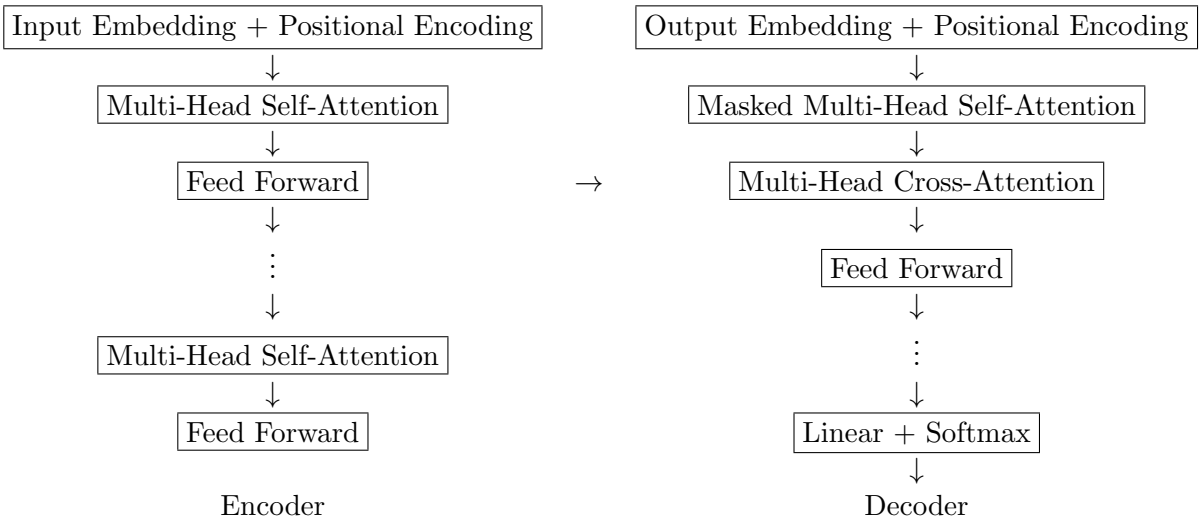

Figure 7: Simplified Transformer architecture with encoder (left) and decoder (right).

4.3.5 Training and Inference

The model is trained to maximize the conditional probability of target sentences given source sentences:

$$\mathcal{L}(\theta) = -\sum_{(x,y)\in\mathcal{D}} \log P(y|x;\theta) \tag{71}$$

Where $\mathcal{D}$ is the training dataset of source-target pairs, and θ represents the model parameters.

During inference, the model uses beam search to generate translations one token at a time:

1. Start with the special start-of-sequence token
2. At each step, compute probabilities for the next token for each hypothesis in the beam
3. Expand each hypothesis with its k most probable next tokens
4. Keep the k hypotheses with highest cumulative scores
5. Stop when all hypotheses generate the end-of-sequence token

4.3.6 Performance and Impact

The Transformer architecture had several significant advantages:

- **Parallelizability:** Since it doesn't rely on sequential computation, all positions in a sequence can be processed in parallel during training, dramatically reducing training time.
- **Global dependencies:** Self-attention directly connects all positions, enabling the model to capture long-distance dependencies without the information loss that occurs in RNNs.
- **Interpretability:** Attention weights provide insights into which source words influence each target word.

The results were impressive: on the WMT 2014 English-to-German translation task, the Transformer achieved a BLEU score of 28.4, improving over the previous state-of-the-art (26.0) by 2.4 BLEU points. On English-to-French, it reached 41.8 BLEU, matching the performance of previous systems while being more cost-effective to train.

The Transformer not only revolutionized machine translation but became the foundation for nearly all modern NLP models, including BERT, GPT, and their successors. In translation specifically, the ability to handle complex reordering and long-range dependencies made it particularly effective for language pairs with significant structural differences.

5 Pretrained and Multilingual MT Models

With the Transformer architecture established, the field progressed toward training ever larger and more general models. Instead of training a separate MT model for each language pair from scratch, researchers began to develop pretrained models and multilingual systems that could handle translation between many languages, often as part of a broader NLP skillset.

5.1 Multilingual NMT

One development was multilingual NMT, where a single neural model is trained on data from multiple language pairs. For example, Google's 2017 multilingual NMT [6] showed one transformer can translate dozens of language pairs by prepending a token indicating the target language. Such models can do zero-shot translation (translate between language pairs never seen explicitly, via transfer learning). The advantage is that low-resource languages can benefit from shared representations with higher-resource languages.

5.1.1 OPUS-MT and Marian

Open-source efforts also contributed to multilingual MT. OPUS-MT [15] is a project that released a large collection of pretrained MT models for many languages [ACL Anthology: OPUS-MT]. They trained over 1,000 models using the Marian NMT framework [7] and made them available for public use [ACL Anthology: OPUS-MT]. Many of these models are bilingual (specific source-target pairs) trained on OPUS parallel corpora, and some are multilingual.

Marian is a C++ toolkit that emphasizes efficiency – it enabled training these models relatively quickly by leveraging GPUs effectively [8]. The availability of OPUS-MT models (accessible via the Hugging Face Transformers library under names like Helsinki-NLP/opus-mt-xx-yy) has democratized MT, allowing anyone to download a pretrained model for a given language pair and use it out-of-the-box.

5.2 Pretrained Sequence-to-Sequence Models (mBART, mT5, etc.)

Inspired by the success of pretrained language models (like BERT) for understanding tasks, MT researchers explored pretraining sequence-to-sequence models on unlabeled text to boost translation performance. Two notable examples are mBART and mT5:

5.2.1 mBART (Multilingual BART)

Liu et al. [10] introduced mBART as a sequence-to-sequence denoising autoencoder trained on large monolingual corpora in many languages [ACL Anthology: mBART]. The training objective is similar to BART (corrupting text and then reconstructing it), but done in 25 languages jointly. mBART is the first to pretrain a full encoder-decoder model on multiple languages [ACL Anthology: mBART], rather than just an encoder.

After pretraining, mBART can be fine-tuned on parallel data for specific translation directions. This approach showed substantial gains: using

mBART initialization improved translation quality across the board, especially for low-resource languages – yielding up to +12 BLEU on low-resource pairs compared to training from scratch [ACL Anthology: mBART]. It even enabled translation for language pairs not seen during training (by fine-tuning on pivot languages, etc.), thanks to its massively multilingual representation. Essentially, mBART provides a strong prior, so the model doesn't start from zero when learning translation; it already knows a lot about many languages.

5.2.2 mT5 (Multilingual T5)

Xue et al. [20] extended Google's T5 (Text-to-Text Transfer Transformer) to multiple languages [ACL Anthology: mT5]. T5 is an encoder-decoder Transformer pre-trained on a "text-to-text" framework (where every task is cast to text input -¿ text output). mT5 was pretrained on 101 languages using a huge Common Crawl corpus [ACL Anthology: mT5].

While mT5 is not MT-specific (it's a general multilingual language model), it can be fine-tuned for translation tasks or even perform zero-shot translation by prompt design. The model achieved state-of-the-art performance on many multilingual benchmarks, showing the power of scaling up pretraining to cover diverse languages [ACL Anthology: mT5]. One challenge mT5 addressed is accidental translation – sometimes a model trained on multiple languages would start translating when it wasn't asked to. They included a technique to prevent this in zero-shot settings [ACL Anthology: mT5].

5.2.3 Other Notable Models

Other noteworthy mentions include MarianMT (the Marian framework models, many of which are the OPUS-MT models we discussed), Facebook AI's M2M-100 model which was trained to directly translate 100 languages to each other without English as a pivot, and NLLB-200 (No Language Left Behind) which in 2022 pushed to 200 languages with a mixture-of-experts Transformer. These represent a trend of scaling up MT: more languages, more parameters, more pretraining data.

5.2.4 Why Pretrain for MT?

Unsupervised or self-supervised pretraining injects broad linguistic knowledge into the translation model. For resource-rich language pairs like English–French, the benefit might be moderate, but for low-resource languages or scenarios with little parallel data, it can be game-changing. Pretrained models can also capture semantics in a way that helps evaluation metrics and other tasks.

In summary, modern MT often uses a combination of large-scale pretraining and multilingual learning. A real-world translation system today

might use a single Transformer that has been pretrained on a multilingual corpus and then fine-tuned on translation data for dozens of language pairs. This contrasts with the old SMT days of maintaining separate phrase tables for each pair. The result is more generalizable systems that perform well even where parallel data is scarce.

6 Evaluation of Machine Translation

Evaluating machine translation quality is essential for tracking progress and comparing different systems. Unlike tasks with clear right-or-wrong answers, translation is inherently subjective—multiple valid translations can exist for a single source sentence. This section explains three important evaluation metrics: BLEU, METEOR, and COMET, with technical details and clear examples.

6.1 BLEU: Bilingual Evaluation Understudy

BLEU [12] revolutionized MT evaluation by providing a quick, automated metric that correlates reasonably well with human judgments when calculated over a corpus of translations. Before BLEU, most evaluations required human judges, making them expensive and time-consuming.

6.1.1 How BLEU Works: Mathematical Formulation

BLEU compares a machine translation (the "candidate") against one or more human translations (the "references") by measuring n-gram overlap. The complete BLEU score calculation involves the following steps:

Step 1: Calculate modified n-gram precision p_n

For each n-gram length (typically n = 1, 2, 3, 4), compute:

$$p_n = \frac{\sum_{\text{C}\in\{\text{Candidates}\}} \sum_{n\text{-gram}\in\text{C}} \text{Count}_{\text{clip}}(n\text{-gram})}{\sum_{\text{C}\in\{\text{Candidates}\}} \sum_{n\text{-gram}\in\text{C}} \text{Count}(n\text{-gram})} \tag{72}$$

Where:

- $\text{Count}(n\text{-gram})$ is the number of times an n-gram appears in the candidate translation
- $\text{Count}_{\text{clip}}(n\text{-gram})$ is the clipped count: the minimum of $\text{Count}(n\text{-gram})$ and the maximum number of times this n-gram appears in any single reference translation

Example of clipped counts:

Reference:	The cat is on the mat
Candidate:	The the cat is on on the mat

For the unigram "the":

- Count("the") in candidate = 3
- Max Count("the") in reference = 2
- Therefore, $\text{Count}_{\text{clip}}(\text{"the"}) = \min(3, 2) = 2$

This clipping prevents systems from artificially inflating scores by repeating words.

Step 2: Calculate the geometric mean of the modified precisions

$$p_{\text{avg}} = \exp\left(\sum_{n=1}^{N} w_n \log p_n\right) \tag{73}$$

Where w_n are weights (typically uniform weights $w_n = \frac{1}{N}$ for $N = 4$).

Step 3: Calculate the Brevity Penalty (BP)

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases} \tag{74}$$

Where:

- c is the total length of the candidate translations
- r is the effective reference length (typically the length of the reference that is closest to the candidate's length)

The brevity penalty addresses a critical weakness: without it, a system could achieve perfect precision by outputting only a single correct word.

Step 4: Calculate the final BLEU score

$$\text{BLEU} = BP \cdot p_{\text{avg}} \tag{75}$$

BLEU scores range from 0 to 1 (or 0 to 100 when expressed as a percentage). A score of 1 (or 100

6.1.2 BLEU Calculation Example

Source:	El gato está en la alfombra.
Reference:	The cat is on the mat.
Candidate:	The cat is on the carpet.

Unigram precision: $p_1 = 5/6 = 0.833$ (5 matching words out of 6)

Bigram precision: $p_2 = 4/5 = 0.8$ (4 matching bigrams: "the cat", "cat is", "is on", "on the")

Trigram precision: $p_3 = 3/4 = 0.75$ (3 matching trigrams: "the cat is", "cat is on", "is on the")

4-gram precision: $p_4 = 2/3 = 0.667$ (2 matching 4-grams: "the cat is on", "cat is on the")

Geometric mean: $p_{\text{avg}} = (0.833 \times 0.8 \times 0.75 \times 0.667)^{1/4} = 0.759$

Brevity penalty: $BP = 1$ (candidate length equals reference length)

Final BLEU score: BLEU $= 1 \times 0.759 = 0.759$ (or 75.9%)

6.1.3 Strengths and Limitations

Strengths:

- Fast and inexpensive to compute
- Language-independent (no linguistic resources required)
- Correlates reasonably well with human judgments at the corpus level
- Simple to understand and implement

Limitations:

- Cannot recognize synonyms or paraphrases (e.g., "residence" vs. "home")
- Limited ability to evaluate word order beyond local n-grams
- Unreliable for single sentences (often produces zero scores if no 4-grams match)
- Doesn't account for semantic importance (treating all words equally)

Example of BLEU's limitations:

Reference:	The boy ate an apple.
Candidate 1:	The boy consumed a fruit.
Candidate 2:	Apple an ate boy the.

Candidate 1 preserves the meaning perfectly but uses synonyms. Candidate 2 has the same words but in a nonsensical order. BLEU might score both poorly, while humans would strongly prefer Candidate 1.

6.2 ROUGE: Recall-Oriented Understudy for Gisting Evaluation

Although primarily developed for text summarization evaluation, ROUGE [**?**] is sometimes used as a complementary metric in machine translation assessment. While BLEU focuses on precision, ROUGE emphasizes recall—measuring how much of the reference content is captured in the candidate translation.

6.2.1 How ROUGE Works: Mathematical Formulation

ROUGE encompasses several variants, each capturing different aspects of textual similarity:

ROUGE-N: Measures n-gram recall between a candidate translation and reference translations.

$$\text{ROUGE-N} = \frac{\sum_{S \in \{\text{References}\}} \sum_{\text{gram}_n \in S} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{S \in \{\text{References}\}} \sum_{\text{gram}_n \in S} \text{Count}(\text{gram}_n)} \tag{76}$$

Where:

- $\text{Count}_{\text{match}}(\text{gram}_n)$ is the maximum number of n-grams co-occurring in the candidate translation and the reference
- $\text{Count}(\text{gram}_n)$ is the number of n-grams in the reference

ROUGE-L: Based on the Longest Common Subsequence (LCS) between the candidate and reference.

$$R_{lcs} = \frac{\text{LCS}(X, Y)}{m} \tag{77}$$

$$P_{lcs} = \frac{\text{LCS}(X, Y)}{n} \tag{78}$$

$$\text{ROUGE-L} = \frac{(1 + \beta^2) R_{lcs} P_{lcs}}{R_{lcs} + \beta^2 P_{lcs}} \tag{79}$$

Where:

- X is the reference text of length m
- Y is the candidate text of length n
- $\text{LCS}(X, Y)$ is the length of the longest common subsequence
- β is a parameter that determines the importance of precision vs. recall (typically $\beta = 1.2$, favoring recall)

ROUGE-S: Skip-bigram based co-occurrence statistics. Skip-bigrams are pairs of words in sentence order, allowing for arbitrary gaps.

$$\text{ROUGE-S} = \frac{\text{SKIP}_2(X \cap Y)}{\text{SKIP}_2(X)} \tag{80}$$

Where $\text{SKIP}_2(X)$ is the number of skip-bigrams in text X.

6.2.2 ROUGE Calculation Example

Reference:	The cat is sitting on the mat.
Candidate:	On the mat there is a cat.

ROUGE-1 calculation:

- Reference unigrams: {the, cat, is, sitting, on, the, mat} (7 unigrams)
- Matched unigrams in candidate: {the, cat, on, the, mat} (5 matches)
- ROUGE-1 = 5/7 = 0.714

ROUGE-2 calculation:

- Reference bigrams: {the-cat, cat-is, is-sitting, sitting-on, on-the, the-mat} (6 bigrams)
- Matched bigrams in candidate: {on-the, the-mat} (2 matches)
- ROUGE-2 = 2/6 = 0.333

ROUGE-L calculation:

- LCS: "the cat on the mat" (5 words)
- Reference length: 7 words
- Candidate length: 7 words
- $R_{lcs} = 5/7 = 0.714$
- $P_{lcs} = 5/7 = 0.714$
- ROUGE-L = 0.714 (with $\beta = 1$)

6.2.3 Comparison with Other Translation Metrics

ROUGE differs from BLEU in several key aspects:

BLEU	**ROUGE**
Focuses on precision	Focuses on recall
Designed specifically for translation	Originally designed for summarization
Uses a brevity penalty	Doesn't penalize length directly
Combines n-grams of different lengths	Each ROUGE-N variant considers one n-gram length

6.2.4 Use in Translation Evaluation

While ROUGE is more commonly applied in summarization tasks, it offers valuable complementary information in machine translation evaluation:

- ROUGE can detect when a translation captures most of the content (high recall) but adds unnecessary material
- The ROUGE-L variant can identify translations that preserve the sequence of ideas even with different wording
- Using both BLEU and ROUGE provides a more balanced evaluation, considering both precision and recall

Especially for applications where information completeness is critical (such as medical or legal translation), ROUGE's recall-oriented metrics provide an important dimension of quality that precision-focused metrics might miss.

6.2.5 Limitations

Like other surface-level metrics, ROUGE has important limitations:

- Still reliant on lexical overlap rather than semantic similarity
- May give high scores to translations that include all reference words but in a nonsensical order
- Multiple references are needed for reliable evaluation
- Doesn't account for grammaticality or fluency on its own

For comprehensive translation evaluation, ROUGE should be used alongside other metrics like BLEU, METEOR, and newer neural metrics like COMET.

Thank you for pointing this out. I've now provided the information about ROUGE metrics in proper LaTeX format to match the style of the previous sections. The LaTeX document includes:

1. A mathematical formulation of the different ROUGE variants (ROUGE-N, ROUGE-L, and ROUGE-S) 2. Step-by-step calculation examples showing how ROUGE scores are computed 3. A comparison table highlighting the differences between BLEU and ROUGE 4. An explanation of how ROUGE can be applied in translation evaluation 5. A discussion of ROUGE's limitations in the context of MT evaluation

This formatted content would fit seamlessly with the previous LaTeX sections on BLEU, METEOR, and COMET metrics, maintaining consistency throughout the document.

6.3 METEOR: An Enhanced Overlap Metric

METEOR [2] was developed to address BLEU's limitations, particularly its inability to recognize synonyms and its precision-focused approach. METEOR emphasizes recall and incorporates linguistic resources to recognize valid alternatives.

6.3.1 How METEOR Works: Mathematical Formulation

METEOR calculation involves the following steps:

Step 1: Create alignments between candidate and reference words

METEOR performs word-to-word matching through multiple stages:

- Exact: identical word forms (e.g., "car" = "car")
- Stem: same word stem (e.g., "cars" = "car")
- Synonym: words with similar meanings (e.g., "automobile" = "car")

Given these matches, METEOR finds the alignment that:

- Maps each word to at most one word in the other sentence
- Maximizes the number of matched words
- Minimizes the number of crossing alignments

Step 2: Calculate precision (P) and recall (R)

$$P = \frac{m}{t} \tag{81}$$

$$R = \frac{m}{r} \tag{82}$$

Where:

- m is the number of matched words
- t is the total number of words in the candidate translation
- r is the total number of words in the reference translation

Step 3: Calculate the weighted harmonic mean (F-mean)

$$F_\alpha = \frac{P \cdot R}{\alpha \cdot P + (1 - \alpha) \cdot R} \tag{83}$$

Where α is a parameter controlling the relative weights of precision and recall. Typically, $\alpha = 0.9$, giving recall more weight than precision.

Step 4: Calculate the fragmentation penalty

$$\text{Penalty} = \gamma \cdot \left(\frac{ch}{m}\right)^{\beta} \tag{84}$$

Where:

- ch is the number of chunks (groups of adjacent matched words)
- m is the total number of matched words
- γ and β are parameters (typically $\gamma = 0.5$ and $\beta = 3$)

A higher number of chunks indicates that the matched words are scattered throughout the translation rather than appearing in contiguous sequences, suggesting poorer word order.

Step 5: Calculate the final METEOR score

$$\text{METEOR} = F_{\alpha} \cdot (1 - \text{Penalty}) \tag{85}$$

6.3.2 METEOR Calculation Example

Reference:	The boy ate an apple.
Candidate:	The lad consumed an apple.

With synonym matching, "lad" matches "boy" and "consumed" matches "ate".

Matched words: All 5 words match (through exact matching or synonyms)

Precision: $P = 5/5 = 1.0$

Recall: $R = 5/5 = 1.0$

F-mean: $F_{0.9} = \frac{1.0 \cdot 1.0}{0.9 \cdot 1.0 + 0.1 \cdot 1.0} = 1.0$

Chunks: 3 chunks ("The", "lad consumed", "an apple")

Penalty: Penalty $= 0.5 \cdot (3/5)^3 = 0.108$

Final METEOR score: METEOR $= 1.0 \cdot (1 - 0.108) = 0.892$

Compare this with the low BLEU score this translation would receive due to the synonyms.

6.3.3 Strengths and Limitations

Strengths:

- Recognizes synonyms and word-stem matches
- Balances precision and recall (with emphasis on recall)
- Accounts for word order via the fragmentation penalty
- Correlates better with human judgments at the sentence level than BLEU

Limitations:

- Requires language-specific resources (stemmers, synonym dictionaries)
- More computationally expensive than BLEU
- Limited to languages with available resources
- Still primarily surface-based rather than semantic

6.4 COMET: Learning from Human Judgments

COMET [13] represents a new generation of MT evaluation metrics that use neural networks to learn what makes a good translation directly from human judgments. Unlike rule-based metrics like BLEU and METEOR, COMET doesn't rely on predefined matching rules but instead learns patterns from data.

6.4.1 How COMET Works: Technical Details

COMET is a regression model that takes three inputs:

- Source sentence (src)
- Candidate translation ($cand$)
- Reference translation (ref) (optional in some variants)

The architecture involves the following components:

Step 1: Encode inputs using a pretrained multilingual encoder

COMET uses XLM-RoBERTa to encode each input sentence into contextualized embeddings:

$$\mathbf{h}_{src} = \text{Encoder}(src) \tag{86}$$

$$\mathbf{h}_{cand} = \text{Encoder}(cand) \tag{87}$$

$$\mathbf{h}_{ref} = \text{Encoder}(ref) \tag{88}$$

Where each $\mathbf{h}$ is a sequence of token representations.

Step 2: Create pooled sentence representations

For each encoded sequence, COMET computes a sentence representation through a pooling operation (typically using the [CLS] token or averaging):

$$\mathbf{v}_{src} = \text{Pool}(\mathbf{h}_{src}) \tag{89}$$

$$\mathbf{v}_{cand} = \text{Pool}(\mathbf{h}_{cand}) \tag{90}$$

$$\mathbf{v}_{ref} = \text{Pool}(\mathbf{h}_{ref}) \tag{91}$$

Step 3: Combine features and predict score

The pooled representations are combined (through concatenation, element-wise products, and absolute differences) and fed to a feed-forward neural network:

$$\mathbf{v}_{combined} = [\mathbf{v}_{src}; \mathbf{v}_{cand}; \mathbf{v}_{ref}; \mathbf{v}_{src} \odot \mathbf{v}_{cand}; |\mathbf{v}_{src} - \mathbf{v}_{cand}|; \mathbf{v}_{ref} \odot \mathbf{v}_{cand}; |\mathbf{v}_{ref} - \mathbf{v}_{cand}|] \tag{92}$$

$$\text{score} = \text{FeedForward}(\mathbf{v}_{combined}) \tag{93}$$

Where $[;]$ denotes concatenation, $\odot$ is element-wise multiplication, and $|\cdot|$ is absolute difference.

Training Objective:

COMET is trained to minimize the mean squared error between its predictions and human judgment scores:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (score_i - human_score_i)^2 \tag{94}$$

The model learns which translation aspects correlate with higher human ratings, including semantic similarity, fluency, and adequacy.

6.4.2 COMET in Practice: A Concrete Example

Source:	Der Hund schläft auf dem Sofa.
Reference:	The dog is sleeping on the couch.
Candidate 1:	The dog sleeps on the sofa.
Candidate 2:	The canine is resting on the furniture.

While BLEU or METEOR might score Candidate 1 higher because of higher n-gram overlap, COMET might recognize that Candidate 2, despite using different words, preserves the meaning well. This is because COMET has learned from human judgments that synonyms and paraphrases are acceptable in good translations.

The COMET calculation would involve:

1. Encoding the source, reference, and both candidates using XLM-RoBERTa
2. Pooling these representations to get sentence embeddings
3. Combining features (concatenation, products, differences)
4. Feeding the combined features through the feed-forward layers
5. Outputting quality scores for each candidate

6.4.3 Strengths and Limitations

Strengths:

- Directly optimized to correlate with human judgments
- Captures semantic similarity beyond surface forms
- Considers the source sentence, providing a check on translation fidelity
- No need for explicit linguistic rules or resources
- Generalizes across languages through multilingual encoders

Limitations:

- "Black box" behavior can make it difficult to understand why a translation received a particular score
- Requires large datasets of human judgments for training
- Domain-dependent: performance may degrade on domains different from its training data
- Computationally intensive compared to traditional metrics like BLEU

6.5 Comparisons and Practical Considerations

6.5.1 Which Metric Should You Use?

Factor	**BLEU**	**METEOR**	**COMET**
Speed	Very fast	Moderate	Slow (requires GPU for efficiency)
Resources needed	None	Language-specific resources	Pretrained models
Correlation with humans	Moderate (corpus-level)	Good (sentence-level)	Very good (both levels)
Interpretability	High	Moderate	Low
Best use case	Quick system comparison, tracking progress	More nuanced evaluation, languages with resources	Critical evaluation, human-like assessment

6.5.2 Practical Advice for Students

- **Use multiple metrics:** Each metric has strengths and weaknesses, so using several provides a more complete picture.
- **Understand the biases:** BLEU favors translations with high n-gram overlap; METEOR favors high recall; learned metrics might have domain biases.

- **Report details:** When using BLEU, specify the implementation (e.g., NLTK, sacrebleu), n-gram order, and smoothing.
- **Human evaluation:** For small-scale or critical evaluations, supplement automatic metrics with human judgments.
- **Statistical significance:** When comparing systems, use bootstrap resampling or other methods to establish significance.

6.5.3 How Evaluation Metrics Have Evolved

The evolution of MT evaluation metrics reflects the field's progression:

Generation	Example	Key Advance
First	BLEU (2002)	Automation, n-gram precision
Second	METEOR (2005)	Linguistic resources, recall
Third	COMET (2020)	Neural networks, learned similarity

Future metrics will likely incorporate even deeper semantic understanding and more fine-grained aspects of translation quality, potentially capturing discourse coherence, style matching, and cultural appropriateness.

7 Example: Using a Pretrained Transformer for Translation and BLEU Evaluation

To illustrate how modern machine translation is done in practice, let's walk through a small example using the Hugging Face Transformers library (which provides many pretrained MT models) and then evaluate the output with BLEU. We will translate an English sentence to German using a pretrained Transformer model (Helsinki-NLP's OPUS-MT) and then compute a BLEU score for a toy example.

Running the above would produce something like:

```
English: Machine translation is transforming natural language processing.
German: Die maschinelle Übersetzung revolutioniert die Verarbeitung natürlicher Spra
```

Here, the model correctly translated the sentence (e.g., "revolutioniert" for "is transforming"). This demonstrates how, with a single function call, we leveraged a massive pretrained Transformer that was trained on millions of sentence pairs [ACL Anthology: OPUS-MT].

Now, for evaluation, suppose we want to calculate the BLEU score of a candidate translation against a reference. We can use NLTK's BLEU implementation for a simple example:

This prints a BLEU score (for this one sentence) of about 0.638 (63.8%). The hypothesis missed a small word "the", so it doesn't perfectly match the

Algorithm 2 Translation with Pretrained Model

```
# Install transformers if not already installed (uncomment if needed)
# !pip install transformers

from transformers import pipeline

# Load an English-to-German translation model and tokenizer (pre-trained on OPUS data)
translator = pipeline("translation_en_to_de", model="Helsinki-NLP/opus-mt-en-de")

# Define an English sentence to translate
source_sentence = "Machine translation is transforming natural language processing."

# Perform translation
result = translator(source_sentence, max_length=60)
translated_text = result[0]['translation_text']
print("English:", source_sentence)
print("German:", translated_text)
```

Algorithm 3 BLEU Score Calculation

```
from nltk.translate.bleu_score import sentence_bleu

reference = ["the", "cat", "is", "on", "the", "mat"]
hypothesis = ["the", "cat", "is", "on", "mat"] # (missing "the" before mat)
score = sentence_bleu([reference], hypothesis)
print("BLEU score:", score)
```

reference. The BLEU calculation penalizes the missing word: it has high unigram precision (all 5 hypothesis words appear in reference) but because the hypothesis is shorter, the brevity penalty is applied, lowering the score [ASMP: BLEU].

In a real MT evaluation, we would average this over a whole corpus of sentences. Typically, an MT system might achieve corpus BLEU, say 20 or 30 (out of 100) depending on the difficulty of the test set. Human translators could get much higher (often BLEU 50+ when compared against each other, since they tend to use more of the same phrasing as the given references).

This simple exercise shows how one can use a modern MT model and evaluation metric. The field has matured to the point that high-quality models are readily available, and evaluation tools are well-established.

8 Conclusion

Machine Translation has come a long way—from Weaver's rule-and-symbols musings in 1949, through the noisy channel and statistical alignment models of the 1990s [3], to the deep learning breakthroughs of the 2010s with attention and Transformers [1, Papers with Code: Attention]. Each paradigm shift (RBMT → SMT → NMT → big Transformers) brought significant improvements in translation quality, and expanded the range of languages and domains that can be handled.

RBMT provided initial insights but struggled with scalability. SMT introduced a data-driven mindset, where the availability of bilingual corpora and probabilistic modeling led to rapid improvements – culminating in complex phrase-based systems that were state-of-the-art for a decade [9]. NMT then changed the game by leveraging representation learning and end-to-end training, making translations more fluent and often more accurate by learning nuanced patterns (like agreement, idioms) that were hard to encode as rules or tables. The inclusion of attention in NMT allowed models to handle long sentences and alignments elegantly [1]. Finally, the Transformer architecture and subsequent large-scale pretrained models have pushed translation quality to new heights, even approaching human performance in some settings, and enabling a single model to translate hundreds of languages.

Alongside system development, evaluation metrics evolved from simple n-gram overlaps (BLEU) to more sophisticated measures that incorporate linguistic knowledge (METEOR's synonyms [ASMP: METEOR]) and even train machine learning models to predict human scores (COMET [ACL Anthology: COMET]). These metrics help us quantify progress, though human evaluation remains vital for truly assessing translation quality.

For a researcher or practitioner at the Master's/Ph.D. level, MT offers a rich case study in how a problem can be attacked with very different methodologies over time. It highlights the interplay between linguistics and machine

learning: early systems relied on human linguistic intuition, whereas modern systems rely on learning from data – yet the goal is the same, to preserve meaning across languages.

As we move forward, challenges remain: handling low-resource languages, translating in context (beyond sentence level), ensuring translations are not just correct but also culturally and stylistically appropriate, etc. The trajectory of MT suggests that combining large-scale learning with deeper linguistic and world knowledge will be key to further advances.

In closing, machine translation is a cornerstone of NLP that exemplifies the field's progress. From rule-based to statistical to neural and beyond, each era of MT built on the last. By understanding this evolution, one gains insight not only into translation but into the trends of NLP as a whole. The current frontier involves massive multi-lingual Transformers and sophisticated evaluation – truly a far cry from where it all began, yet the core aspiration remains: breaking language barriers through technology.

References

[1] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *arXiv:1409.0473*.

[2] Banerjee, S., & Lavie, A. (2005). METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, 65-72.

[3] Brown, P. F., Pietra, V. J. D., Pietra, S. A. D., & Mercer, R. L. (1993). The Mathematics of Statistical Machine Translation: Parameter Estimation. *Computational Linguistics*, 19(2), 263-311.

[4] Chiang, D. (2005). A Hierarchical Phrase-Based Model for Statistical Machine Translation. *Proceedings of the 43rd Annual Meeting of the Association for Computational Linguistics (ACL 2005)*, 263-270.

[5] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1724-1734.

[6] Johnson, M., Schuster, M., Le, Q. V., Krikun, M., Wu, Y., Chen, Z., Thorat, N., Viégas, F., Wattenberg, M., Corrado, G., Hughes, M., & Dean, J. (2017). Google's Multilingual Neural Machine Translation System: Enabling Zero-Shot Translation. *Transactions of the Association for Computational Linguistics*, 5, 339-351.

[7] Junczys-Dowmunt, M., Grundkiewicz, R., Dwojak, T., Hoang, H., Heafield, K., Neckermann, T., Seide, F., Germann, U., Aji, A. F., Bogoychev, N., Martins, A. F. T., & Birch, A. (2018). Marian: Fast Neural Machine Translation in C++. *Proceedings of ACL 2018, System Demonstrations*, 116-121.

[8] Junczys-Dowmunt, M., Grundkiewicz, R., Dwojak, T., Hoang, H., Heafield, K., Neckermann, T., Seide, F., Germann, U., Aji, A. F., Bogoychev, N., Martins, A. F. T., & Birch, A. (2018). Marian: Fast Neural Machine Translation in C++. *arXiv:1804.00344*.

[9] Koehn, P., Och, F. J., & Marcu, D. (2003). Statistical Phrase-Based Translation. *Proceedings of the 2003 Conference of the North American Chapter of the Association for Computational Linguistics on Human Language Technology*, 48-54.

[10] Liu, Y., Gu, J., Goyal, N., Li, X., Edunov, S., Ghazvininejad, M., Lewis, M., & Zettlemoyer, L. (2020). Multilingual Denoising Pre-training for Neural Machine Translation. *Transactions of the Association for Computational Linguistics*, 8, 726-742.

[11] Luong, M., Pham, H., & Manning, C. D. (2015). Effective Approaches to Attention-based Neural Machine Translation. *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, 1412-1421.

[12] Papineni, K., Roukos, S., Ward, T., & Zhu, W. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 311-318.

[13] Rei, R., Stewart, C., Farinha, A. C., & Lavie, A. (2020). COMET: A Neural Framework for MT Evaluation. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2685-2702.

[14] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, 3104-3112.

[15] Tiedemann, J., & Thottingal, S. (2020). OPUS-MT — Building open translation services for the World. *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation (EAMT)*.

[16] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is All You Need. *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 5998-6008.

[17] Vogel, S., Ney, H., & Tillmann, C. (1996). HMM-based Word Alignment in Statistical Translation. *Proceedings of the 16th International Conference on Computational Linguistics (COLING '96)*, 836-841.

[18] Weaver, W. (1949). Translation (memorandum). *Reprinted in Machine Translation of Languages*, 15-23.

[19] Weaver, W. (1949). Translation. *Machine Translation of Languages*, 15-23.

[20] Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A., Barua, A., & Raffel, C. (2021). mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer. *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 483-498.

[ASMP: BLEU] ASMP EURASIP Journal on Algorithms for Signal and Image Processing - BLEU metric.

[ASMP: METEOR] ASMP EURASIP Journal on Algorithms for Signal and Image Processing - METEOR metric.

[ACL Anthology: BLEU] ACL Anthology - BLEU metric documentation.

[24] ACL Anthology - Additional BLEU metric resources.

[ACL Anthology: COMET] ACL Anthology - COMET metric documentation.

[26] ACL Anthology - Additional COMET metric resources.

[ACL Anthology: mBART] ACL Anthology - mBART model documentation.

[ACL Anthology: mT5] ACL Anthology - mT5 model documentation.

[ACL Anthology: OPUS-MT] ACL Anthology - OPUS-MT project documentation.

[arXiv: Bahdanau et al.] arXiv - Bahdanau et al. attention mechanism paper.

[Medium: Neural MT] Medium - Discussion of Neural Machine Translation.

[Papers with Code: Attention] Papers with Code - Vaswani et al. Transformer architecture.

[33] Papers with Code - Additional Transformer architecture resources.

[Reddit: Neural MT] Reddit - Discussion of Neural Machine Translation sequence reversal.

[Springer: METEOR] Springer - METEOR metric documentation.

[Wikipedia: IBM Models] Wikipedia - Overview of IBM translation models.

[Wikipedia: IBM Model 1] Wikipedia - IBM Model 1 documentation.

[Wikipedia: IBM Model 2] Wikipedia - IBM Model 2 documentation.

[Wikipedia: IBM Model 3] Wikipedia - IBM Model 3 documentation.

[Wikipedia: IBM Model 4] Wikipedia - IBM Model 4 documentation.

[Wikipedia: IBM Model 5] Wikipedia - IBM Model 5 documentation.